

LOOPGYM: TRAINING FOR COMBINED INFERENCE IN TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Loop-invariant generation is compositional: one response may contain a useful bound, another a relational equality, and neither need prove the target alone. This makes pass@1 a poor training and evaluation objective for an inference procedure that can combine partial answers. We study this problem under *target-hidden loop verification*, where the model sees the precondition, guard, and body but not the assertion or postcondition Q . Our inference procedure samples multiple responses, splits conjunctive invariants into clauses, unions them, iteratively removes syntax failures, and uses verifier-checked Houdini pruning to retain an inductive subset before Q is restored for the final proof. We align both stages of training with this rollout-combine-filter procedure. First, synthetic supervised targets are formed by applying it to multiple target-hidden responses and serializing the combined verified set, thereby compressing a multi-response search result into one response. Second, reinforcement learning rewards two properties needed by the same inference procedure: the behavioral strength of each response and its complementary contribution within a rollout group. The reward uses reachable executions and conservatively filtered synthetic negative examples, never Q . The resulting objective is not to maximize standalone proof rate, but to train responses that combine into a stronger verified set under a fixed inference budget. On 832 C programs, five-rollout inference verifies 74.76% of tasks, compared with 61.78% for the strongest external baseline. Reinforcement learning raises combine@10 from 52.40% to 56.97% even as pass@1 decreases, and reduces the rollouts needed to reach 95% of the measured maximum from 30 to 10. Thus ten trained-model responses outperform the combination of one hundred base-model responses.

1 INTRODUCTION

Verifying any program with a loop requires finding an adequate invariant: no proof can otherwise cover arbitrarily many iterations, and writing invariants by hand remains a central bottleneck to verifying real software. Loop invariants summarize an unbounded computation in a finite assertion. They must be inductive while retaining enough program knowledge to establish the desired property at loop exit. A weak predicate such as *true* can satisfy inductiveness but prove nothing useful. We formalize soundness, logical strength, and target sufficiency in Section 2.

Invariant generation is compositional: one response may provide a bound and another a relational equality, so their union can prove a target that neither proves alone. Pass@1 discards this partial progress. We instead sample multiple responses and retain a verified subset of their combined clauses with our rollout-combine-filter framework (Section 4.2).

This procedure needs both *strength* and *exploration*: each response should contain useful clauses, while different responses should cover different relations. Optimizing standalone success instead encourages repetition and ignores partial answers. Our objective is the filtered combination at a fixed rollout budget, ideally compressing large-budget exploration into one practical deployment run.

We study this alignment problem under *target-hidden loop verification*, where the target is withheld until after the invariant set has been fixed (Section 2). Existing methods can use the target, failed obligations, or counterexamples to guide search (Gulwani et al., 2008; Gupta & Rybalchenko, 2009; Dillig et al., 2013; Alur et al., 2013; Garg et al., 2014; 2016; Si et al., 2020; Kamath et al., 2023; Cao

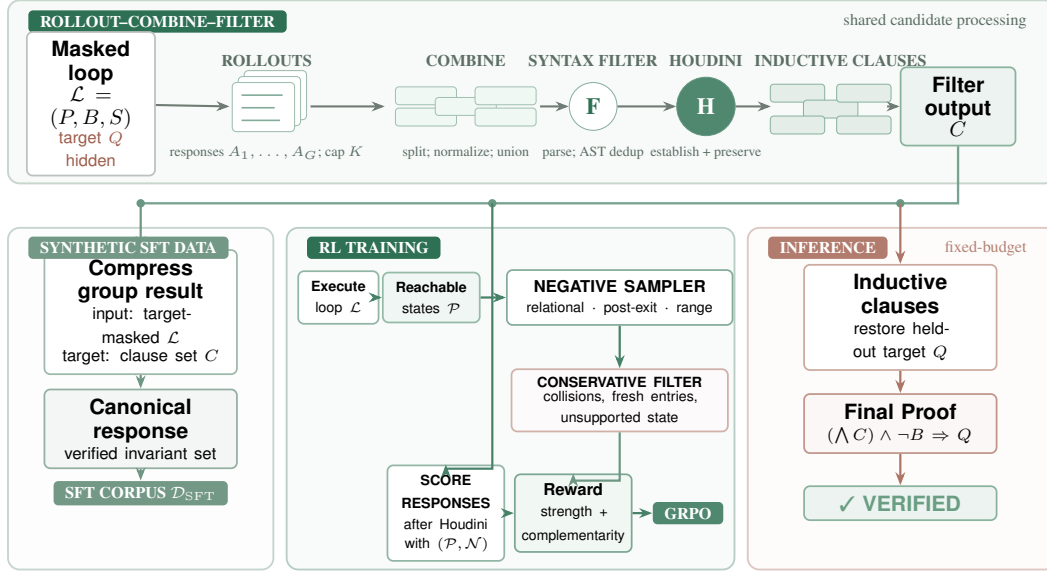


Figure 1: **Training is organized around rollout-combine-filter inference.** Multiple target-hidden responses are split into clauses, capped, normalized, and combined. The filter iteratively removes syntax failures, optionally performs AST deduplication, then uses Houdini to iteratively remove clauses that fail establishment or preservation. SFT compresses the result into one supervised response; RL rewards strong and complementary responses. Deployment restores Q only after filtering.

et al., 2025). Hiding Q prevents copying an inductive postcondition. Final success is unchanged: the verifier must prove soundness and sufficiency for the restored target.

LOOPGYM aligns both training stages with this structure. For SFT, it applies the same framework, then serializes the verified set as one target for the masked program. This distills multi-response search into a stronger response for later combined inference.

Within loop-invariant generation specifically, prior systems search, filter counterexamples, or rank candidates, but none trains a generation policy with reinforcement learning (Section 3). A usable reward is itself hard to define once Q is hidden: binary inductiveness cannot separate a useful invariant from a weak one such as *true*, and querying Q during training would defeat the target-hidden setting. For RL, executions provide reachable states and conservative negative examples without querying Q . After Houdini, negative coverage rewards response strength; a group-relative bonus gives more credit to behavior not covered by peer responses. Together they approximate the strength and complementarity needed after combination.

Our experiments deliberately separate standalone success from combined inference. On 832 integer C programs, five target-hidden responses with union and Houdini verify 622 programs (74.76%), compared with 514 (61.78%) for the strongest external baseline. On Qwen3-8B, reinforcement learning decreases direct pass@1 from 9.30% to 4.88% but increases combine@10 from 52.40% to 56.97%. Ten responses from the trained policy exceed combine@100 of the base model, reducing the rollouts needed to reach 95% of the measured maximum from 30 to 10. Training thus compresses broad base-model exploration into a much smaller deployment budget rather than optimizing one standalone target-proving answer.

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification as a compositional generation problem and evaluate the filtered combination of multiple responses, rather than treating pass@1 as the deployment objective.

- We align training with rollout–combine–filter inference. Synthetic SFT targets distill filtered combinations into individual responses, while an execution-based RL reward balances per-response strength with complementary contributions across a rollout group without observing Q .
- We show that this alignment improves both combined verification and rollout efficiency, and audit the synthetic negative examples that support the reward.

2 PRELIMINARIES

2.1 LOOP VERIFICATION

A verification task consists of a loop $\mathcal{L} = (P, B, S)$ and a postcondition Q , where P is the precondition, B is the loop guard, and S is the loop body. Let $\text{Reach}(\mathcal{L})$ denote the states reachable at the loop head, including the final guard test at loop exit.

An invariant I is *sound* (equivalently, inductive) when it is established by P and preserved by one iteration:

$$P \Rightarrow I, \quad \{I \wedge B\} S \{I\}.$$

These obligations ensure that I holds on every state in $\text{Reach}(\mathcal{L})$ (Floyd, 1967; Hoare, 1969).

We say that I_1 is *stronger* than I_2 , written $I_1 \succeq I_2$, when

$$\models I_1 \Rightarrow I_2.$$

An invariant I^* is *exact* when

$$\llbracket I^* \rrbracket = \text{Reach}(\mathcal{L}).$$

Because every sound invariant contains all reachable states, an exact invariant is stronger than every sound invariant of the same loop:

$$I^* \text{ exact}, J \text{ sound} \implies \models I^* \Rightarrow J.$$

Exactness is a semantic ideal; the reachable states need not have a finite representation in the annotation fragment used by the generator.

Finally, a sound invariant I is *Q -sufficient* when it proves the postcondition at loop exit:

$$I \wedge \neg B \Rightarrow Q.$$

Thus exactness describes the loop independently of a target, whereas Q -sufficiency is relative to one postcondition. The final verifier must prove both soundness obligations and this exit obligation.

2.2 TARGET-HIDDEN VERIFICATION

In *target-hidden loop verification*, the generator and all intermediate feedback see $\mathcal{L} = (P, B, S)$ but not Q . The postcondition is restored only after the generated invariant has been fixed. Preconditions and executable loop semantics remain visible, while assertions, postconditions, assertion calls, and conventional error-label names are masked. The untouched program is used only for final verification.

2.3 SCOPE AND MOTIVATING EXAMPLE

We focus on numerical loops for two reasons. First, their invariants are predominantly arithmetic equalities and inequalities, including polynomial relations. These predicates are comparatively amenable to automatic checking, which provides a scalable and objective verification signal. Second, numerical loops factor out the representational machinery of arrays, heaps, and recursive data structures, whose verification often requires auxiliary functions or inductive predicates. This gives a controlled setting in which success depends more directly on discovering algebraic relations and reasoning about loop dynamics.

Concretely, we study one braced `while` loop over scalar C integers, including nonlinear updates; arrays, heaps, and recursive structures are out of scope. This restriction does not make invariant discovery trivial: even a small loop can hide a nonlinear relation across coupled recurrences.

Consider the following abridged program:

162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215

```
1 int x = 1, y = 1, c = 1;  
2 while (c < k) { c++; x = x*z + 1; y = y*z; }  
3 /*@ assert 1+x*z-x-z*y == 0; */
```

The visible assertion can be copied verbatim as an invariant; when hidden, its nonlinear relation must instead be recovered from the coupled updates.

3 RELATED WORK

Programmatic invariant synthesis. Classical methods infer invariants with abstract interpretation, affine equalities, templates, interpolation, recurrences, or constraint solving (Karr, 1976; Cousot & Cousot, 1977; Cousot & Halbwachs, 1978; Sankaranarayanan et al., 2004; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017). These methods provide strong guarantees for selected languages, especially affine transition systems, but nonlinear templates trade generality for increasingly difficult algebraic search. Dynamic approaches instead infer likely relations from execution traces and then filter or prove them (Ernst et al., 2007; Nguyen et al., 2012; Garg et al., 2016). LOOPGYM also uses executions, but only to construct a training signal; final soundness is established independently by the verifier. Target-directed software model checkers constitute a related but distinct setting. For example, ULTIMATE AUTOMIZER (UAuto) uses trace abstraction and interpolation to verify a supplied safety property (Heizmann et al., 2013). Although its correctness witnesses may contain inductive assertions, they are synthesized with the verification target visible. UAuto therefore does not address our target-hidden setting, in which the assertion is withheld throughout invariant generation, and we do not include it in the experimental comparison.

Neural and LLM-based invariant generation. Learned methods broaden the candidate language by using neural generators, counterexamples, or verifier feedback (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020). LOOPY generates candidate clauses, filters them with Houdini, and repairs failures using verifier feedback (Kamath et al., 2023). CLAUSE2INV combines generated clauses under counterexample guidance, and IRANK learns to rank LLM-generated invariants before expensive verifier calls (Cao et al., 2025; Chakraborty et al., 2023). These systems solve a supplied target and commonly evaluate whether one response proves it. We hide the target during generation and training, and train for the filtered combination of compatible clauses from multiple responses.

Specification generation. LLM-based tools also synthesize broader program specifications. AUTOSPEC repeatedly proposes and verifies annotations; SPECGEN combines LLM proposals with mutation in Java/JML; and SESPEC uses symbolic execution to generate goal-independent specifications for C (Wen et al., 2024; Ma et al., 2025; Yang et al., 2026). We study only loop invariants and ask whether they prove a held-out target, rather than evaluating a general behavioral specification.

Reinforcement learning from formal feedback. Verifier-guided reinforcement learning has been used for program synthesis and formal reasoning. RE:FORM uses scalable curation and verifier-guided RL for Dafny, while SPECRL generates impossible input–output pairs and rewards specifications that reject them (Yan et al., 2025; Huang et al., 2026). Validity alone does not distinguish a useful specification from a weak one. We specialize this idea to loops: synthetic negative states measure strength, while the verifier proves inductiveness. Verifiable-reward RL can also improve small-budget accuracy while narrowing extensive-sampling coverage (Yue et al., 2025). Because deployment unions samples, our reward adds group-relative credit for less common behavior that survives Houdini.

4 METHOD

4.1 OVERVIEW

Using the notation of Section 2, the model receives only the target-masked loop $\mathcal{L}$. We parse each response, split every invariant at its top-level conjunctions, normalize the resulting clauses, and obtain

```

216   Input: masked loop  $\mathcal{L}$ , responses  $A_{1:G}$ , cap  $K$ , AST-dedup flag  $d$ 
217   Output: filtered clause set  $C$ 
218   1:  $C \leftarrow \bigcup_{i=1}^G \text{Cap}_K(\text{Normalize}(\text{Split}_\wedge(A_i)))$ 
219   2: repeat
220   3:    $E \leftarrow \text{ParserErrors}(\text{Annotate}(\mathcal{L}, C))$ 
221   4:    $C \leftarrow C \setminus E$ 
222   5: until  $E = \emptyset$  or  $C = \emptyset$ 
223   6: if  $C = \emptyset$  then return  $C$ 
224   7: if  $d$  then  $C \leftarrow \text{ASTDedup}(C)$ 
225   8: repeat
226   9:    $V \leftarrow \text{WP}_{\text{est,pres}}(\mathcal{L}, C)$ 
227   10:   $D \leftarrow \{c \in C : \neg V_{\text{est}}(c) \vee \neg V_{\text{pres}}(c)\}$ 
228   11:   $C \leftarrow C \setminus D$ 
229   12: until  $D = \emptyset$  or  $C = \emptyset$ 
230   13: return  $C$ 

```

Algorithm 1: Rollout–combine–filter

an ordered list $A = (a_1, \dots, a_m)$. We cap each response at K clauses,

$$\bar{A} = (a_1, \dots, a_{\min(m, K)}), \quad o(A) = \max(0, m - K),$$

to prevent a response from gaining coverage by producing an unbounded list.

Our *rollout–combine–filter* framework is shared by data construction and deployment. It samples responses, combines their clauses, iteratively removes syntax failures, optionally performs AST deduplication, and then applies Houdini. SFT turns the filtered combination into one target, RL improves the responses that populate a combination, and inference applies the same framework before restoring Q .

4.2 ROLLOUT–COMBINE–FILTER

Rollout. The model emits ACSL (Baudin et al., 2021) annotations of the form `loop invariant <expr>;`. We split each `<expr>` at top-level logical conjunctions, so a conjunction contributes separate clauses that can survive or fail independently. Splitting precedes capping and normalization.

Combine. Given G responses, we union their clause lists:

$$X_0 = \bigcup_{i=1}^G \bar{A}_i.$$

This combination preserves partial progress across responses but can contain syntactically invalid or non-inductive clauses.

Filter. Algorithm 1 gives the full framework. The filter first iteratively removes clauses identified by the verifier’s parser or type checker. It may then deduplicate the syntax-valid clauses by their normalized ASTs. Finally, Houdini iteratively removes every clause whose establishment or preservation obligation fails under the verifier (Flanagan & Leino, 2001). We denote the surviving set by $\text{Filter}_{\mathcal{L}}(X)$. Filtering can retain compatible clauses from imperfect responses but cannot invent a relation absent from the combined pool.

During RL scoring, sampled reachable states provide a cheap pre-filter for out-of-scope or obviously false clauses, after which the verifier remains the source of soundness. Inference omits this optimization; all uses otherwise share clause splitting, capping, normalization, combination, syntax filtering, and Houdini.

4.3 COMPRESSING MULTI-RESPONSE SEARCH INTO SFT TARGETS

For every masked loop $\mathcal{L}$, we sample a response group, combine its clauses, and compute $\text{Filter}_{\mathcal{L}}(X)$. A nonempty result is serialized as one canonical response:

$$\mathcal{D}_{\text{SFT}} = \{(\mathcal{L}, \text{serialize}(\text{Filter}_{\mathcal{L}}(X)))\}.$$

This distills a filtered multi-response result into one answer. Neither side contains Q : the input is masked and the output contains only inductive clauses. Execution states are used only for RL, never as SFT answers.

4.4 EXECUTION-BASED TRAINING EXAMPLES

To rank inductive sets without revealing Q , we derive a strength signal from executions. The sampler instruments one braced `while` loop, compiles it with `gcc`, and records loop-head valuations for a deterministic, precondition-checked input sweep. The recorded states form $\mathcal{P} \subseteq \text{Reach}(\mathcal{L})$; clauses falsified by these states are removed before scoring. If any run hits the iteration cap, the sample is marked incomplete and cannot support terminal or range examples.

The sampler then constructs three kinds of negative examples without reading an assertion or postcondition:

- **Relational perturbations** change one or two variables by $\pm\{1, 2\}$ in densely observed states, testing relations such as $x + y = n$.
- **Post-exit continuations** execute the real loop body for additional steps after a genuine exit.
- **Range perturbations** move one variable outside its observed range and test useful bounds.

Figure 2 illustrates how the three families are generated from observed executions and then converted into negative traces.

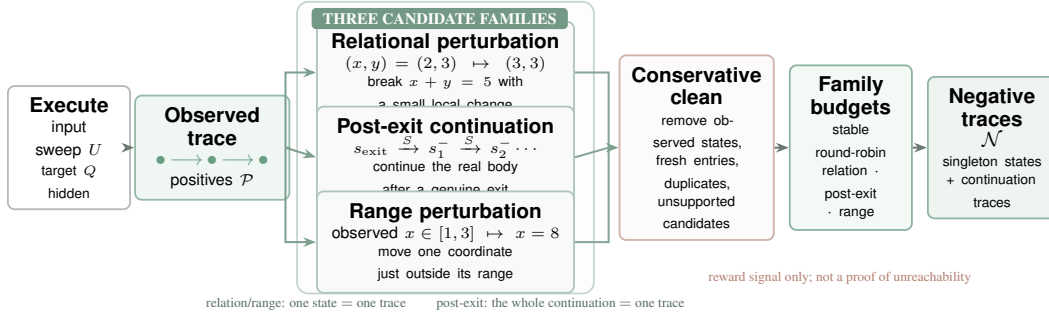


Figure 2: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Three transformations probe relational structure, behavior past a genuine exit, and sampled bounds. Conservative cleaning and per-family budgets produce $\mathcal{N}$, which is used only for RL reward computation and never exposes Q .

These are training signals, not proofs of global unreachability. We remove observed reachable states, possible fresh loop entries, duplicates, and candidates that depend on unsupported state. Relation and range candidates form singleton negative traces, whereas all states from one post-exit continuation form one trace. Each family has an independent budget, filled round-robin across variable and direction buckets. Pointers, arrays, calls, unsupported locals, or oracle-dependent behavior disable the affected construction; a program with no safe variable contributes positives only. Capped executions contribute positives but disable constructions that require complete terminal or range information. Algorithm 2 in Appendix A.1 and Table 2 give the sampling procedure and exact configuration.

Let $\mathcal{N}$ denote the retained negative traces. One trace can contain several witness states from a perturbed execution. For a candidate set X , let $\text{rej}(X) \subseteq \mathcal{N}$ be the traces on which at least one clause in $\text{Filter}_{\mathcal{L}}(X)$ is false.

4.5 REWARDING STRENGTH AND COMPLEMENTARITY

The deployed union needs strong individual responses that remain complementary within a group. We reward both properties.

Soundness is checkable directly: Houdini either establishes and preserves a clause or it does not. Strength is not. An exact invariant need not have a finite representation in our annotation fragment (Section 2), so we cannot target exactness itself, and a binary sound/unsound label cannot tell a response that is merely inductive from one that is inductive and useful: *true* passes the same check as a tight bound. We therefore need a graded, target-independent proxy for strength that a partial, sound response can satisfy by degree rather than all at once.

For response A_i , let $R_i = \text{rej}(\bar{A}_i)$ after standalone filtering. Its negative rejection rate is

$$\text{coverage}(A_i) = \frac{|R_i|}{|\mathcal{N}|}.$$

This gives dense credit to a useful partial answer without querying the hidden target.

Coverage alone can cause every response to repeat the easiest high-reward clauses. For a group of G responses, define

$$f(n) = \sum_{j=1}^G \mathbf{1}[n \in R_j], \quad (1)$$

$$\text{bonus}(A_i) = \frac{1}{|\mathcal{N}|} \sum_{n \in R_i} \left(1 - \frac{f(n)}{G}\right). \quad (2)$$

A rejection receives less additional credit when most responses already provide it. The bonus is therefore a target-independent proxy for complementary value after union, while coverage retains absolute strength.

We penalize only conservative duplicates within a response,

$$d_{\text{dup}}(A) = |\bar{A}| - |\text{dedup}(\bar{A})|.$$

The key normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association, but avoids transformations requiring stronger arithmetic assumptions. Unique zero-coverage clauses are not penalized because they may support another clause or the hidden target.

The generation reward is

$$r_{\text{gen}}(A_i) = w_c \text{coverage}(A_i) + w_g \text{bonus}(A_i) - w_d d_{\text{dup}}(A_i) - w_o o(A_i), \quad (3)$$

with $(w_c, w_g, w_d, w_o) = (1.0, 0.3, 0.02, 0.05)$. Clauses beyond K receive no coverage and incur overflow cost. Houdini removes failures before scoring, so one bad clause does not erase useful clauses in the response.

We also compute the rejection rate of $\text{Filter}_{\mathcal{L}}(X_0)$ for the combined pool. It controls group re-sampling but is not copied to every response. With no safe negatives, reward falls back to binary inductiveness for a nonempty response.

Group-relative policy update. Each masked loop $\mathcal{L}$ samples a group of G token sequences $o_{1:G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{L})$ from the current policy; parsing, splitting, and capping (§4.2) turn o_i into the scored response A_i . We normalize r_{gen} within its own sampling group to obtain a group-relative advantage,

$$\hat{A}_i = \frac{r_{\text{gen}}(A_i) - \text{mean}_{1 \leq j \leq G} r_{\text{gen}}(A_j)}{\text{std}_{1 \leq j \leq G} r_{\text{gen}}(A_j) + 10^{-6}},$$

shared by every token of o_i , and update θ with the clipped GRPO objective (Shao et al., 2024)

$$\mathcal{J}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right], \quad (4)$$

where $\rho_{i,t}(\theta) = \pi_\theta(o_{i,t} \mid \mathcal{L}, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid \mathcal{L}, o_{i,<t})$ and π_{ref} is the frozen reference policy.

Group-relative normalization is why the bonus term matters for training, not only for evaluation. Coverage alone is bounded, so once sampling collapses onto one easiest clause every response in the group reports nearly the same reward, the group standard deviation shrinks, and $\hat{A}_i$ becomes an uninformative, high-variance signal for every member of the group, exactly the collapse a per-response objective invites. The bonus breaks these ties before normalization: a response that only repeats a peer’s rejections is pushed toward $\hat{A}_i \approx 0$ even though its raw coverage is positive, while a response exposing a rejection no peer in the group produced keeps a positive advantage. Group-relative policy optimization therefore rewards a rollout for what it adds to its own group, which is the same quantity that determines whether the rollout helps combine@ k .

4.6 DEPLOYMENT AND THE FIXED-BUDGET OBJECTIVE

At deployment, the model samples k masked responses, splits and normalizes their clauses, combines them, and runs the syntax and Houdini filters. No execution examples are constructed. Only then are the retained clauses and Q inserted into the untouched program for the final verifier proof.

The retained clause set and the invariant predicate produced by one inference invocation are therefore

$$X_k = \bigcup_{i=1}^k \bar{A}_i, \quad C_k = \text{Filter}_{\mathcal{L}}(X_k), \quad I_k = \bigwedge_{a \in C_k} a, \quad A_i \sim \pi(\cdot \mid \mathcal{L}),$$

rather than the best standalone response. SFT compresses a large candidate pool into stronger responses; RL improves their coverage and complementarity. Combine@ k and k_{95} measure how few trained-policy responses are needed to match a much larger base-model sample.

5 EXPERIMENTS

We ask: (1) how much union and Houdini improve target-hidden inference, (2) whether RL improves combined inference rather than merely pass@1, and (3) whether the gains reflect clause complementarity and a reliable reward.

5.1 EXPERIMENTAL SETUP

Benchmarks. We evaluate 832 integer C programs. The *Core-366* subset contains 316 linear programs from Code2Inv, SyGuS, and SV-COMP and 50 nonlinear programs from LIPuS. The remaining 466 programs are the integer fragment of Loopy’s original 469-program corpus; three floating-point programs are excluded because they fall outside our scalar-integer predicate language. Unsupported inputs remain failures in the common denominator.

Target-hidden evaluation. Before any model-facing step, we remove `assert` and `ensures` predicates, executable assertion calls, and conventional error-label names. Preconditions and executable loop semantics remain visible. After a method fixes its invariants, they are inserted into the untouched original program. A task succeeds only if the source parses, the verifier proves establishment and preservation for every retained invariant, and all restored target goals succeed. Failures, timeouts, malformed outputs, and unsupported programs count as zero.

Verifier, models, and baselines. All final checks use Frama-C 31.0 (Gallium) (Kirchner et al., 2015), WP, and Z3 4.13.3. The main inference comparison fixes `gpt-5-nano` across all LOOPGYM variants, so differences among those rows come only from inference. We compare with the native target-hidden pipelines of AUTOSPEC, SESPEC, and CLAUSE2INV, and with Daikon 5.8.24 (Wen et al., 2024; Yang et al., 2026; Cao et al., 2025; Ernst et al., 2007). CLAUSE2INV supports only the 366 programs with transition verification conditions; its remaining inputs still count as failures.

The model-scale analysis samples 100 target-hidden responses per task from the official Qwen3-4B, Qwen3-8B, Qwen3-14B, and Qwen3-30B-A3B checkpoints (Yang et al., 2025), at temperature 1.0 with thinking disabled. The reinforcement-learning comparison uses Qwen3-8B and the corresponding 8B-RL-Zero checkpoint under the same 100-response protocol.

Metrics. For a program with c independently successful responses among $n = 100$, direct pass@ k uses the unbiased estimator $1 - \binom{n-c}{k} / \binom{n}{k}$. Combine@ k instead unions the clauses from the first k responses, applies Houdini, and tests the resulting invariant set against the restored target. Direct pass@ k asks whether one response is independently Q -sufficient; combine@ k asks whether the accumulated, filtered union is Q -sufficient and therefore measures deployment. We summarize compression of exploration with

$$k_{95} = \min\{k : C_{\pi}(k) \geq 0.95 C_{\pi}(100)\},$$

where $C_{\pi}(k)$ is combine@ k for policy π .

5.2 RQ1: HOW MUCH DOES COMBINED INFERENCE HELP?

NAIVE uses one response and a minimal prompt. LOOPGYM (pass@1) tests one response from the LOOPGYM prompt as a standalone invariant set. LOOPGYM (combine@1), LOOPGYM (combine@5), and LOOPGYM (combine@10) combine and filter clauses from one, five, or ten target-hidden responses, respectively. Retries are disabled in this comparison.

Method	Linear / 316	NLA / 50	Loopy / 466	All / 832	Time (s)	Avg. tokens
AUTOSPEC	208	4	302	514 (61.78%)	54.77	25,154.54
CLAUSE2INV	63	1	0	64 (7.69%)	29.04	385.80
DAIKON	73	0	101	174 (20.91%)	4.89	0
SESPEC	180	3	231	414 (49.76%)	117.31	44,699.37
NAIVE	154	4	221	379 (45.55%)	28.87	4,493.37
LOOPGYM (pass@1)	193	7	252	452 (54.33%)	53.52	6,928.30
LOOPGYM (combine@1)	203	7	294	504 (60.58%)	66.60	6,928.30
LOOPGYM (combine@5)	250	7	365	622 (74.76%)	284.81	33,404.56
LOOPGYM (combine@10)	255	7	376	638 (76.68%)	536.81	68,953.35

Table 1: Target-hidden verification results and mean generation cost per task. Tokens are averaged over tasks on which the method is called. We use API-reported totals when available and tokenizer-based estimates from saved outputs otherwise. Pass@1 and combine@1 reuse the same single rollout and therefore have identical model-token cost; their time differs only because combine@1 additionally runs the local filter. Unsupported Clause2Inv inputs remain failures in the 832-program denominator but invoke no model.

Five-response inference verifies 622 programs, 108 more than the strongest external baseline. The controlled sequence is 379/832 for NAIVE, 452/832 for LOOPGYM (pass@1), 504/832 for LOOPGYM (combine@1), and 622/832 for LOOPGYM (combine@5); the last setting lets clauses from five responses jointly form a filtered proof.

Ten responses add only 16 proofs over five while roughly doubling tokens and time. This diminishing return motivates a model whose small union contains clauses the base model reveals only under wider sampling.

5.3 RQ2: DOES RL ALIGN THE MODEL WITH COMBINED INFERENCE?

We compare Qwen3-8B with 8B-RL-Zero while holding the candidate union, Houdini filter, target-hiding protocol, and response budget fixed.

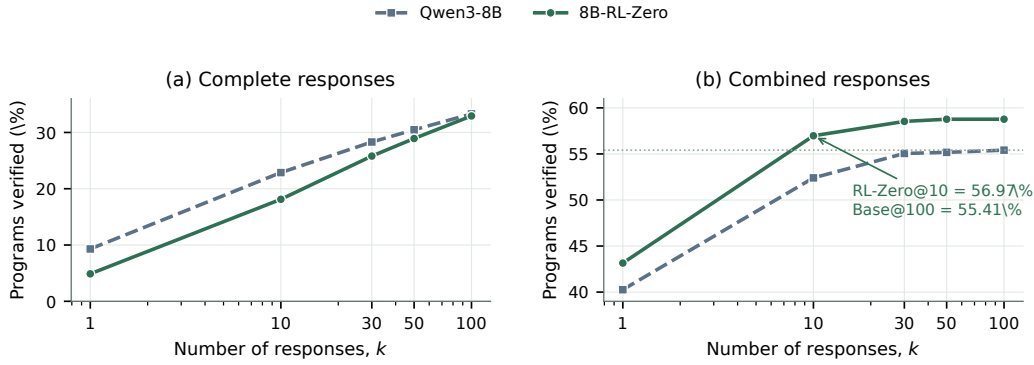


Figure 3: Reinforcement learning improves combined inference even though it does not improve the chance that one response proves the target. With ten responses, 8B-RL-Zero exceeds the base model’s combine@100 result.

Figure 3 shows different trends for standalone target-proving responses and complementary clauses. RL-Zero lowers direct pass@1 from 9.30% to 4.88% and leaves direct pass@100 nearly unchanged (33.29% versus 32.93%). Under the deployed combination procedure, however, combine@10 rises from 52.40% to 56.97%, and combine@100 rises from 55.41% to 58.77%. Ten RL-Zero responses outperform 100 base-model responses, while k_{95} falls from 30 to 10.

The pass@1 decrease is not a failure under the deployment objective: one ten-response invocation exceeds the 100-response base result. Coverage strengthens each response, while the group bonus rewards behavior missed by peers even when no response is independently Q -sufficient.

5.4 RQ3: WHERE DOES THE COMPOSITIONAL GAIN COME FROM?

Partial answers remain useful after filtering. Inductiveness by itself cannot distinguish a useful invariant from true, and a binary whole-response reward discards useful clauses when another clause in the same response fails. Reinforcement learning with verifiable rewards can also concentrate probability on already rewarded paths while reducing large- k exploration (Yue et al., 2025). Union and Houdini expose the value that these objectives miss: valid clauses survive even when the response containing them does not prove the target on its own.

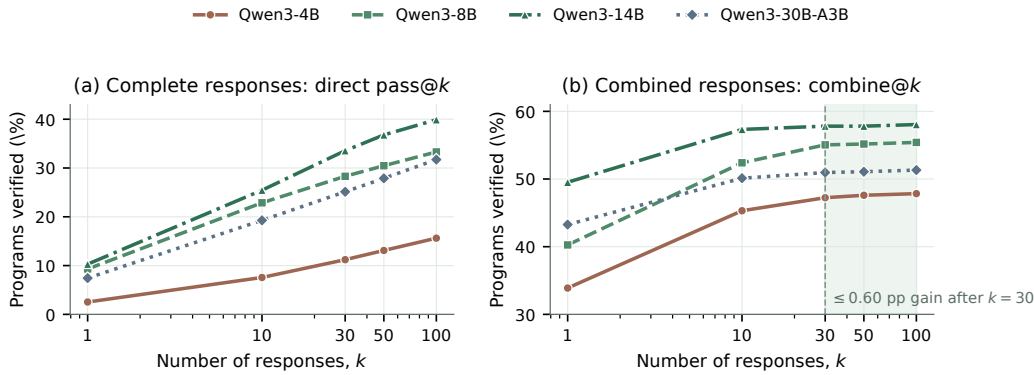


Figure 4: Direct pass@ k measures standalone target-proving responses, whereas combine@ k measures whether the accumulated clauses contain a Q -sufficient Houdini-filtered subset. For all four official Qwen3 checkpoints, the combine curves change by at most 0.60 percentage points after $k = 30$.

Figure 4 shows that partial responses contain substantial value. Direct pass@100 is 15.62% for Qwen3-4B, 33.29% for 8B, and 39.90% for 14B. Combining clauses raises 4B to 47.84% and 14B to

58.05%. Houdini can recover compatible clauses from imperfect responses, but it cannot invent a relation that the model never emits. The gap between direct $\text{pass}@k$ and $\text{combine}@k$ is evidence that responses contain reusable partial proofs rather than only all-or-nothing solutions.

No checkpoint gains more than 0.60 points from $\text{combine}@30$ to $\text{combine}@100$. Reward must therefore strengthen early responses while preserving complementarity, not rely on unbounded sampling.

Reward and negative-example checks. Across ten hand-written specifications, every gold invariant survives the verifier filter and rejects at least 99.6% of the intended negative examples, with no ordering violations in the reward sanity checks. A larger audit on Core-366 compares 486,302 generated negative states against an expanded set of program executions and finds no hard pair-level collision. These checks do not prove global unreachability, but support the conservative construction. With the opposite movement of $\text{pass}@1$ and $\text{combine}@k$, they support interpreting the reward as a signal for strong, complementary clauses rather than hidden- target labels.

6 CONCLUSION AND LIMITATIONS

Because clauses compose across responses, our objective is the filtered combination, not $\text{pass}@1$. Inference uses rollout–combine–filter before restoring the target. SFT distills a verified combination into one response; target-independent RL rewards response strength and behavior not covered by peers. Consequently, ten trained-policy responses outperform one hundred base-model responses even as $\text{pass}@1$ decreases.

The implementation supports one scalar-integer loop. Negative examples are training signals, the group bonus approximates contribution after union, and soundness depends on the verifier. Multiple loops and richer state remain future work (Liu et al., 2024).

REFERENCES

- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.
- Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL: ANSI/ISO C specification language. <https://frama-c.com/html/acsl.html>, 2021.
- Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.
- Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.
- Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proc. 5th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 84–96, 1978.
- Isil Dillig, Thomas Dillig, Boyang Li, and Ken McMillan. Inductive invariant generation via abductive inference. In *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 443–456, 2013.

-
- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.
- Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.
- Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.
- Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.
- Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 499–512, 2016.
- Sumit Gulwani, Saurabh Srivastava, and Ramarathnam Venkatesan. Program analysis as constraint solving. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 281–292, 2008.
- Ashutosh Gupta and Andrey Rybalchenko. InvGen: An efficient invariant generator. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 634–640. Springer, 2009.
- Matthias Heizmann, Jochen Hoenicke, and Andreas Podelski. Software model checking for people who love automata. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, volume 8044 of *Lecture Notes in Computer Science*, pp. 36–52. Springer, 2013. doi: 10.1007/978-3-642-39799-8_2.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.
- Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.
- Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.
- Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.
- Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.
- Chang Liu, Xiwei Wu, Yuan Feng, Qinxiong Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.
- Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.
- ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pp. 683–693, 2012.

-
- Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.
- Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. Non-linear loop invariant generation using Gröbner bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 318–329, 2004.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.
- Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.
- Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.
- Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026.
- Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

A IMPLEMENTATION DETAILS

Table 2 lists the implemented defaults. Execution sampling is used only to compute reinforcement-learning rewards; it does not construct the synthetic SFT corpus. SFT synthesis, RL, and inference share the rollout–combine–filter implementation: the same conjunction splitting, response cap, normalization, candidate combination, iterative syntax filter, optional AST deduplication, and Houdini pruning. SFT serializes the filtered combination, RL scores the responses that populate it, and inference restores Q only after filtering. Inference does not construct execution examples.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
range-perturbation steps	$\pm\{5, 8, 13, 21, 34\}$	candidates kept only outside the sampled range
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budgets	$320 (\leq 64 \text{ fallback}) / 64 / 128$	relational / post-exit / range examples; stable round-robin within each kind
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	(1.0, 0.3)	negative rejection / group-relative bonus weights
w_d	0.02	cost per conservative duplicate; unique zero-coverage clauses are not penalized
K	20 clauses	response cap applied independently to every generated response
w_o	0.05	overflow cost per clause after K
re-roll threshold	0.6	group batch score below which re-rolling is advised
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Z3, 30 s, Typed model	per-goal prover budget

Table 2: Defaults for execution sampling (§4.4), reward computation (§4.5), inference, and verification.

The exact near-input tier order is 0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9.

A.1 NEGATIVE-TRACE SAMPLING

Algorithm 2 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. $\text{Perturb}_{1,2}$ applies the configured single-variable and pairwise changes, while RangeEscape moves one variable just outside its sampled range. The exact perturbations and budgets are those in Table 2.

```

756   Input: masked loop  $\mathcal{L} = (P, B, S)$ , input schedule  $U$ , configuration  $\Theta$ 
757   Output: reachable states  $\mathcal{P}$ , negative traces  $\mathcal{N}$ 
758   1:  $\mathcal{T}, \mathcal{C}_{\text{post}}, c \leftarrow \text{Trace}(\mathcal{L}, U, \Theta) \triangleright c: \text{some run was capped}$ 
759   2:  $\mathcal{P} \leftarrow \text{UniqueHeads}(\mathcal{T}); \quad M \leftarrow \text{SafeModifiedVars}(S)$ 
760   3: if  $M = \emptyset$  or  $S$  has unsupported state then return  $(\mathcal{P}, \emptyset)$ 
761   4:  $\mathcal{S} \leftarrow \text{StratifiedSample}(\mathcal{P})$ 
762   5:  $C_{\text{rel}} \leftarrow \text{Perturb}_{1,2}(\text{Dense}(\mathcal{S}), M, \Theta)$ 
763   6: if  $\neg c$  then  $C_{\text{rel}} \leftarrow C_{\text{rel}} \cup \text{Perturb}_{1,2}(\text{Terminals}(\mathcal{T}), M, \Theta)$ 
764   7:  $C_{\text{range}} \leftarrow \text{RangeEscape}(\mathcal{S}, M, \mathcal{P}, \Theta)$  if  $\neg c$ , else  $\emptyset$ 
765   8:  $(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}) \leftarrow \text{Clean}_{\mathcal{P}}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}})$ 
766   9:  $N_{\text{rel}}, N_{\text{post}}, N_{\text{range}} \leftarrow \text{BudgetAndRoundRobin}(C_{\text{rel}}, \mathcal{C}_{\text{post}}, C_{\text{range}}, \Theta)$ 
767   10:  $\mathcal{N} \leftarrow \{\langle s \rangle : s \in N_{\text{rel}}\} \cup N_{\text{post}} \cup \{\langle s \rangle : s \in N_{\text{range}}\}$ 
768   11: return  $(\mathcal{P}, \mathcal{N})$ 

```

Algorithm 2: Target-hidden negative-trace sampling

$\text{Clean}_{\mathcal{P}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates, using family priority relation, post-exit, then range. A cleaned post-exit continuation remains one trace; relation and range candidates become singleton traces. Within the relation budget, candidates that preserve the guard truth value and remain inside the sampled coordinate ranges are selected first.

B GENERATION PROMPT

The same generation template proposes the responses later combined during SFT synthesis, RL, and inference. Any assertion is stripped before the program is inserted. General annotation rules are supplied separately by `system_prompt.txt`.

Generation prompt

You are given a C function. Produce the strongest inductive ACSL loop invariants that capture the loop's behavior (conservation/relational laws + tight progress bounds). Output ONLY invariant lines, each formatted exactly as:
 loop invariant <expr>;
 Output at most 20 invariant lines.

Program:
 {program}

C TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and Z3. It exposes `POST /reward`, accepting program source and an array of model responses. The implementation retains the following field names for compatibility:

```

{program, n_positives, n_negatives, filter_mode,
 reward_mode, rollout_rewards[], base[], hard_bonus[],
 redundant_clauses[], redundancy_penalty[],
 generated[], accepted[], overflow[], overflow_penalty[],
 rollouts[], batch_score, should_reroll}

```

Responses may be raw LLM text, invariant lists, or annotated code; parsing is uniform. An offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.